Pointers

Pointers and Addresses

- A pointer is a variable that contains the address of a variable.
- Use of pointers lead to more compact and efficient code.
- Pointers and arrays are closely related.
- Pointers + goto: recipe for complex code.
- Dangling references.
- With discipline, pointers can be used to achieve clarity and simplicity.

Pointers and Addresses

- A typical machine has consecutively addressed memory cells.
- A byte can be a `char`, two adjacent bytes can form a `short integer`, and four adjacent bytes can form a `long`.
- A pointer is a group of cells that can hold an address.

Pointers and Addresses

```
char *p;
```

It says that the expression `*p` is a char

```
char c;
p = &c; /* p now points to c */
```

Pointers and Addresses

- The unary operator & gives the address of an object.
- The unary operator * is the indirection or dereferencing operator.
- & applies only to objects in memory: variables, array elements. Not to expressions, constants, or register variables.

```
int x = 1, y = 2, z[10];
int *ip; /* ip is a pointer to int */
ip = &x; /* ip now points to x */
y = *ip; /* y is now 1 */
*ip = 0; /* x is now 0 */
ip = &z[0]; /* ip now points to z[0] */
```

Pointers and Addresses

- The unary operators * and & bind more tightly than arithmetic operators.
- unary operators like * and ++ associate right to left.

```
double *dp, atof(char *);
int x = 1, y = 2;
int *ip = &x;
y = *ip + 1; /* adds 1 to *ip, and assigns to y */
*ip += 1;    /* increments what ip points to */
++*ip;       /* increments what ip points to */
(*ip)++;     /* increments what ip points to */
*ip++;  /* increments ip; applies * on the new ip */
```

Pointers and Addresses

- Since pointers are variables, they can be used without dereferencing.

```
int *ip, *iq;
...
iq = ip; /* copies the contents of ip into iq */
```

Pointers and Function Arguments

- C passes function arguments by value.
- Direct alteration of variables in the calling function is not possible.
- Example: Sorting routine using a function called `swap`.

Pointers and Function Arguments

```
/* Incorrect swap function */
void swap(int x, int y)
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}

/* Correct swap function using pointers */
void swap(int *px, int *py)
{
    int temp;
    temp = *px;
    *px = *py;
    *py = temp;
}
```

Data Structures

Data Structures

What are Data Structures?

Data structures are ways of organizing and storing data in a computer so that it can be used efficiently. They define the relationship between data and the operations that can be performed on the data.

- Data structures are crucial for writing efficient algorithms.
- They provide a means to manage large datasets and optimize access and manipulation of data.
- There are various types of data structures, each with its own strengths and weaknesses.

Common Data Structures

Some common data structures include arrays, linked lists, stacks, queues, trees, graphs, hash tables, etc.

Unsorted Array

Unsorted Array

Introduction

- Array data structure where the elements are not arranged in any specific order.
- Elements are stored in the order in which they are inserted or added to the array.
- Searching for a particular element is not efficient compared to a sorted array.

Unsorted Array

Operations

- **Insert:** Add a new element to the end of the array.
- **Search:** Iterate linearly through the array to find the desired element.
- **Delete (specified with pointer):** Replace with the last element and adjust the array length.
- **Delete (specified with key):** Find the key, replace with the last element, and adjust the array length.
- **FindMin:** Iterate to find the minimum element.
- **DeleteMin:** Find the minimum element's index, replace with the last element, and adjust the array length.
- **DecreaseKey:** Decrease the value of the element specified by pointer.
- **Meld:** Concatenate two unsorted arrays.

Unsorted Array

Time bounds

If n is the current size of the data structure,

- **Insert:** $O(1)$
- **Search:** $O(n)$
- **Delete (specified with pointer):** $O(1)$.
- **Delete (specified with key):** $O(n)$.
- **FindMin:** $O(n)$.
- **DeleteMin:** $O(n)$.
- **DecreaseKey:** $O(1)$.
- **Meld:** $O(n)$.

Unsorted Array

Implementation of Insert

```
// Function to insert an element into the array
void insert(int arr[], int *psize, int key) {
    if (*psize < MAX_SIZE)
        arr[(*psize)++] = key;
    else
        printf("Error: Array is full\n");
}
```

Unsorted Array

Implementation of Search

```
// Linear search
// Function to search for an element in the array
int search(int arr[], int size, int key) {
    for (int i = 0; i < size; i++)
        if (arr[i] == key)
            return i;
    return -1; // Element not found
}
```

Unsorted Array

Implementation of deleteByPointer

```
// Function to delete an element specified with a pointer
void deleteByPointer(int arr[], int *psize, int index) {
    if (index >= 0 && index < *psize)
        arr[index] = arr[--*psize];
    else
        printf("Error: Invalid index\n");
}
```

Unsorted Array

Implementation of deleteByKey

```
// Function to delete an element specified with a key
void deleteByKey(int arr[], int *psize, int key) {
    int index = search(key);
    if (index != -1)
        arr[index] = arr[--*psize];
    else
        printf("Error: Element not found\n");
}
```

Unsorted Array

Implementation of findMin

```
// Function to find the minimum element in the array
int findMin(int arr[], int size) {
    if (size == 0) {
        printf("Error: Array is empty\n");
        return -1;
    }

    int min = arr[0];
    for (int i = 1; i < size; i++)
        if (arr[i] < min)
            min = arr[i];
    return min;
}
```

Unsorted Array

Implementation of deleteMin

```
// Function to delete the minimum element in the array
void deleteMin(int arr[], int *psize) {
    if (*psize == 0) {
        printf("Error: Array is empty\n");
        return;
    }

    int minIndex = 0;
    for (int i = 1; i < *psize; i++)
        if (arr[i] < arr[minIndex])
            minIndex = i;

    arr[minIndex] = arr[--*psize];
}
```

Unsorted Array

Implementation of decreaseKey

```
// Function to decrease the key of an element at a specified index
void decreaseKey(int arr[], int size, int index, int newValue) {
    if (index >= 0 && index < size && arr[index] > newValue)
        arr[index] = newValue;
    else
        printf("Error: Invalid index\n");
}
```

Unsorted Array

Implementation of meld

```
// Function to merge two arrays (Meld operation)
void meld(int array1[], int *psize1, int array2[], int size2) {
    if (*psize1 + size2 <= MAX_SIZE) {
        for (int i = 0; i < size2; i++)
            insert(array1[], psize1, array2[i]);
    } else
        printf("Error: Meld would exceed array size limit\n");
}
```

Unsorted Array

Example Usage in `main()`

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

int main() {

    int arr[MAX_SIZE], size = 0;

    insert(arr, &size, 5);  insert(arr, &size, 2);
    insert(arr, &size, 8);  insert(arr, &size, 1);
    printf("Array: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

    int keyToSearch = 8;
    int searchResult = search(arr, size, keyToSearch);
    if (searchResult != -1)
        printf("%d found at index %d\n", keyToSearch, searchResult);
    else
        printf("%d not found\n", keyToSearch);
```

Unsorted Array

Example Usage in `main()`

```
deleteByPointer(arr, &size, 1);
printf("Array after deleting element at index 1: ");
for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

deleteByKey(arr, &size, 2);
printf("Array after deleting element with key 2: ");
for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

printf("Minimum element in the array: %d\n", findMin(arr, size));

deleteMin(arr, &size);
```

Unsorted Array

Example Usage in `main()`

```
    printf("Array after deleting the minimum element: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

    decreaseKey(arr, size, 1, 0);
    printf("Array after decreasing the key at index 1 to 0: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

    int array2[] = {3, 7, 4};
    meld(arr, &size, array2, sizeof(array2) / sizeof(array2[0]));
    printf("Array after melding with another array: ");
    for (int i = 0; i < size; i++) printf("%d ", arr[i]); printf("\n");

    return 0;
}
```

Sorted Array

Sorted Array

Introduction

- Array data structure where the elements are arranged either in ascending or in descending order.
- Searching for a particular element is efficient, as binary search can be employed.

Sorted Array

Operations: Overview

- **Search:** Binary search for finding specific elements.
- **Insert:** Shift the larger elements to the right by one position and insert the element.
- **Delete (specified with pointer):** Remove the specified element and potentially shift others.
- **Delete (specified with key):** Use binary search to locate the key and adjust the array.
- **FindMin:** The minimum element is always at the first index.
- **DeleteMin:** Remove the element at the first index and shift remaining elements.
- **DecreaseKey:** Delete the specified element and reinsert it with decreased key.
- **Meld:** Merge two sorted arrays into a single sorted array.

Sorted Array

Time bounds

If n is the current size of the data structure,

- **Search:** $O(\log n)$
- **Insert:** $O(n)$
- **Delete (specified with pointer):** $O(n)$.
- **Delete (specified with key):** $O(n)$.
- **FindMin:** $O(1)$.
- **DeleteMin:** $O(n)$.
- **DecreaseKey:** $O(n)$.
- **Meld:** $O(n)$.